Should you use AI to help you code

Augustana University

Marcus Birkenkrahe

August 24, 2026

An update to my note on using AI to write code (2026)

My views on AI for beginning coders have not changed much since 2024 (see below) - but my own workflow (as a CS/DS teacher) has changed, and I have much more experience with students now.

I have mixed news: AI can be useful even to beginning coders if you use it discerningly - at the right time in the right way, but it has design limitations that will never go away (unless the underlying architecture changes profoundly, which may happen, but as of August 2026 that's not the case).

I will update the list of use cases below at irregular intervals. These uses are strictly speaking only for non-agentic, or conversational AI (that means that the AI only knows what you copy into the conversation, and can only give you output in that conversation).

I have published my more recent insights on using AI for research and on teaching agentic AI to beginners - see birkenkrahe.github.io.

What if you are looking for an answer to a question?

When you use AI as a **conversational search tool**, you need to ask AI to be specific and provide its sources. Example:

> What does immutability in Java mean? Give me a brief answer with a single example, and give me a validated source.

You can see the answer (from Gemini) here. It is not guaranteed that the cited sources exist - so you will potentially lose time tracking them down (which you must do).

However: It is **much** better to look this up yourself using existing online documentation (or a book), and construct an example yourself, because it trains you to search yourself, identify good sources, and teach you to read existing documentation. And if you can come up with your own examples, you learn to write code much faster than if you only use pre-fabricated examples.

What if you ask the same question twice?

Results are strictly non-replicable because AI is non-deterministic. Instead, it is stochastic, which means you won't know what exactly it will return (and that is true for everybody, being an expert does not change it). You may not care about this now when you're only interested in one answer, and when your problems are simple and when you work alone, but when you become a professional, all of these things will change dramatically, and you will feel the need to obtain **reliable** results. AI is not reliable by design.

What if you need a drill?

When you come back to a task after a break, or when you're not sure if you understood something after you read it even several times, you can use AI as a useful tutor. Begin by loading it with all the information that you have digested yourself, and ask it to drill you using only that information. You may have to reign it in a little, for example:

> "Ask me only one questions at a time, only based on the material that I gave you. Never just give me the solution. When I fail to give the correct answer, ask me follow up questions until I get the correct answer. Mix conceptual and coding questions. [etc.]

I have found this very useful especially when I was juggling multiple topics and was likely to forget some or all of what I thought I had learnt. The same caveat applies as always: the AI may hallucinate (lie to you) - but in this case you should already know what the correct answer is so you're not so vulnerable.

What if you use AI for a programming assignment?

This is the most difficult use case for me to judge because of my own experience: it is hard to put myself back into your place as a total beginner. However, I am also frequently still learning new things, I am baffled by what

I see, and I don't know what to do, and my knowledge of the tools is limited, sometimes severely so.

Here is my workflow (only for assignments where AI use is permitted). Unfortunately, it is not short, because assignments are supposed to be hard and test your mettle:

1. You should always try to find a solution on your own first.

2. When you are stuck, give yourself a little more time, or perhaps a lot more if you feel that your pride is on the line.

3. When you're really stumped, try to talk to a human first. This may get you unstuck: explain the problem to the human (that sometimes already gets you moving again).

4. If there's no human available, you can use AI, but always sparingly: make sure to tell the AI that you do not want the solution to the problem, and be specific about what you (think you) need. At this point, it will become clear to you if you even understand what the problem is. A good approach is to explain to the AI what you think the issue is and ask it to check your logic (or your pseudocode).

5. Unfortunately, the latest generative AI is very fast and very good at finding errors in the typical learner problems (because they're not all that hard - you should be able to solve them in a couple of hours perhaps).

6. At some point, with the help of AI, you'll have your solution, except it's not actually yours, is it? Fortunately, you know this and since you don't code for a grade but because you want to grow as a programmer, now comes the hardest step.

7. You need to consider this problem unsolved by you - though you may submit it (as long as you disclose that and how exactly you used AI). You need to ask the AI for another problem similar to it but not too much (this will require some experimenting). This new problem you must solve entirely on your own.

8. If you needed AI support for the new problem, too, you still haven't achieved much (though, incidentally, you did learn something, about yourself at the very least, and probably some coding, too).

9. You now need to ask for a third problem, and explain to the AI what your results and your process were, for example:

   > I failed to solve two problems on my own, without your help. These were my difficulties in particular: [list them if you can]. To improve, give me a third problem that tests these difficulties in particular.

10. Let's assume you solved it on your own, finally, and you feel good now (though not as good as if you had done it by yourself). Notice that it is totally different from looking through documentation, books or using stackoverflow.com because then you still need to integrate what you find to build something on your own (which is often the most difficult step), and you need to debug it and perhaps find a better way, too.

Looking at this, and comparing it with your own process, which may be different, you may appreciate that, especially for small problems in school that were designed for you to solve on your own, the use of AI is overkill - it is a waste of precious time if you could have solve it; and it is unlikely that you could not have solved it (otherwise the fault lies with your teacher who should have altered the problem).

By the way, AI is particularly crap at efficient coding though it is often good at owning up to its own deficiencies, except that it will wait for you to point them out ("Hold on, is this really the only way?")

The new Python textbook Think Python by Allen B. Downey that we use for CS I has exercises that suggest to you to "ask a virtual assistant", and he does a good job steering clear of naive use of generative AI.

A note on using AI to write code for you or debug your code (2024)

Question from a recent Lyon College graduate who now works as a software engineer, who asked for my recommendation regarding use of AI:

"I was wondering if you could give me any guidance. We are changing the way that we store stuff in our warehouse. They asked me if it would be possible to run some code with an AI to see what it would say. I asked ChatGPT and it gave me a pretty simple solution which involved getting the size, frequency grabbed, etc."

What do you think? Let's hear your answers:

1. Should a software engineer use AI to get ideas or improve his code?

2. Are there any potential issues with this approach?

3. Do you think AI will change software engineering, and how?

4. Should you use AI to let it help you learn in technical classes?

My answers (short version):

1. He should only use AI if he does not care about the result - if he does not depend on it, and if he has ample time to waste.

2. AI lies unpredictably, often delivers misguided output, makes you forget stuff you already knew, and gives you a false sense of security. It's like having a slave who really wants to help but is incapable of having even a single thought of his own.

3. It has already changed software engineering though there are no good studies yet, only a lot of calls for research. The pace of development of these principally intransparent tools is too fast.

4. No unless you have a lot of time to waste and/or already know your stuff really well (and even then, it's dangerous).

Here is my very long answer to the guy for the record:

Use AI for things you really don't care about, and if you don't care how much time you waste, because while the code it produces can be debugged, you will not know when and where the AI "hallucinates" i.e. spews nonsense. As a result, it may be hard to debug since you cannot talk to it about its own mistakes as you would to a coworker. AI aims to please you at any cost and has no insight or understanding of what it does at all. Syntax errors are less likely, version and dependency errors are frequent (since it doesn't test its own code). Logic errors are brutal and hard to find and you may spend precious time dealing with the AI that you could have used to learn something new or test or create yourself (which would have taught you something). If you keep using it for things that are easy, you'll forget what you once knew. It's worse than copy-and-paste because it will always try to give you the whole thing, and copy-and-paste programming at least forces you to think the code through that you copied (ideally anyway). It is probably OK (haven't tried it) when it comes to web stuff since web stuff is easy. But it'll likely fail when you try it on anything that's complex (of course it has got better, too, over the past year, but only marginally, in this regard). In terms of raw code, its solutions are often not as easy as they

could be, use the wrong data structures, have no concern for performance, and generally will mess with your own style - without offering anything in return (like a better or different style, which would happen if you worked with a human master programmer). Slight changes here depending on the language: C/C++ have lots of issues, Python probably the best (most public code examples I presume), SQL and R are pretty good (because they're simple and well-documented).

Having said that, I use it a lot (as I say in the talk) but only for stuff I don't care about very much, and also I am often not a slave to deadlines like you, and I have decades more experience to cut through the BS that I'm given. I use it when I'm lazy in the full knowledge that I may have to start all over, and to give me an alternative (which sometimes works), or sometimes because I'm lonely (unlike you I have nobody who programs with or alongside me). When AI teaches me something new, I have to re-learn it just as if I had heard it in a lecture or in a video (and check carefully).

I have spoken about this at length at Oklahoma University last March (link).

Some recent evidence in a home debugging story (2025):

I set up a new workstation at home with a Linux distro that I had not used before. After installing Emacs from scratch, I got the error message: `"Invalid function: org-assert-version"`, which relates to a function in the Org-mode package and disabled many other commands at once. ChatGPT's advice was lengthy, involved re-installing software packages and peppering the Emacs configuration file with new functions. I put all of these suggestions to work since I didn't want to think about it - I just wanted to be done with (after many hours of installing things, late at night). The next morning I did not open the AI and just looked at the files that Emacs complained about, and that coud not be processed. I immediately saw that there was an empty line where Emacs did not expect to find one (Org-roam expects to find an ID in the first line of the file). I had inadvertently created that empty line earlier when editing something. This is the old way - thinking about errors, getting frustrated, taking the error to bed and mulling it over, sometimes for hours, while doing something else in the meantime. Then, always, the answer "presents itself", or rather, it is generated by your wetware, your beautiful, God-given brain.

Short summary: At your stage, using AI is a waste of time at best, and a crime against your ability to learn at worst. Learning never comes without pain and (temporary) desperation. AI is like a pill but one that only works some of the time, and you'll never know when. Instead: join Lyon's Programming Student Club and experience the pain of not knowing

first hand every week!

You don't have to believe me - experiment with it on your own but make sure that you don't get addicted to the convenience of AI.

Will you be punished for using AI in my class?

Not directly because nobody can tell if you used AI or not but indirectly by turning in suboptimal results, by learning less, and by having less time for other, more productive activities.

Are there any data on this?

Not much on coding per se but a recent (06/24), substantive, long (59 p) paper titled "Generative AI Can Harm Learning"), based on a very carefully conducted field experiment with a large (1000) sample of high school students concluded: "Our results suggest that students attempt to use [AI] as a "crutch" during practice problem sessions, and when successful, perform worse on their own. Thus, to maintain long-term productivity, we must be cautious when deploying generative AI to ensure humans continue to learn critical skills." (Bastani et al, 2024).

Here are two recent accounts from coders: Schluntz (07/24) / Carlini (08/24)

References

Bastani, Hamsa and Bastani, Osbert and Sungu, Alp and Ge, Haosen and Kabakcı, Özge and Mariman, Rei, Generative AI Can Harm Learning (July 15, 2024). Available at ssrn.com.

Carlini, How I Use "AI" (August 1, 2024). Available at carlini.com.

Schluntz, Replacing my Right Hand with AI (July 30, 2024). Available at erikschluntz.com.